

MAP ATIVIDADE 01

Aluna: Suelle Ferreira Maciel

Questões Discursivas:

- **Qual problema o Strategy resolve?** O padrão Strategy resolve o problema do acoplamento ao separar os diferentes algoritmos de cálculo de frete em classes específicas. Assim, a classe responsável pelo pedido não precisa conhecer os detalhes de cada cálculo, ela só delega essa tarefa para a estratégia selecionada, permitindo que a forma de envio seja alterada dinamicamente (em tempo de execução).
- **E se usássemos só if/else?** Teríamos uma classe que não ia parar de crescer a cada nova regra de frete, tornando a manutenção insustentável. Além disso, acabaríamos duplicando a lógica de negócio caso outras partes do sistema também precisassem calcular fretes, e a criação de testes ficaria muito mais complexa, já que não conseguiríamos testar cada regra de forma isolada.
- **Como o sistema fica aberto para extensão?** O sistema atende ao Princípio Aberto-Fechado porque, para adicionar um novo tipo de frete, basta criarmos uma nova classe que implemente a interface FreteStrategy (como a classe FreteInternacional). Isso é feito sem precisar alterar nada de código na classe Pedido ou nas estratégias que já existem.
- **E se cada país tiver uma regra diferente?** Cada país se tornaria uma estratégia independente. Teríamos que criar classes específicas e separadas para cada um, como FreteInternacionalUapao.java e FreteInternacionalCuba.java, por exemplo.
- **Outros cenários para Strategy:** Pensando em um sistema de vendas, o padrão Strategy também seria uma boa escolha para gerenciar as diferentes formas de pagamento (pix, cartão, boleto), pra calcular descontos baseados no método de pagamento escolhido, ou pra processar o cálculo de impostos.

Desafio Extra:

- **São novas estratégias ou promoções?** São promoções. Elas não definem uma nova forma base de calcular o frete, em vez disso, elas apenas atuam modificando ou complementando o resultado de uma estratégia de frete que já existe.

- **Deveriam modificar Sedex, PAC, etc?** Não. Se adicionássemos a regra de gratuidade (como um `if valor > 399`) diretamente dentro das classes de envio, estaríamos duplicando código em todas elas. Isso exigiria a modificação de muitas classes sempre que uma nova promoção fosse criada ou alterada, o que é uma má prática.
- **Como evitar alterar as classes existentes?** Podemos criar a classe `FreteGratisAcimaDe` que implementa a interface `FreteStrategy`. A lógica é verificar se a compra ultrapassa R\$ 399, se sim, ela retorna 0 de frete, caso contrário, ela delega a responsabilidade do cálculo para a estratégia original. Essa abordagem funciona em conjunto com qualquer modalidade de frete (ex: `new FreteGratisAcimaDe(new Sedex(), valorCompra, 399)`) e permite até que várias promoções sejam combinadas